MONKDB WORKBENCH · ENGINEERING

Comprehensive end-to-end test suite — SQL compatibility & browser UI

REPORT DATE	MONKDB VERSION	PLAYWRIGHT
2026-03-02	6.0.0-SNAPSHOT	1.58.2 · Chromium
TOTAL TESTS	**SQL COMPAT**	**BROWSER UI**
68	37 tests	31 tests

✓ All 68 tests passing — 0 failures

68	68	37	31
TOTAL TESTS	PASSED	SQL COMPAT	BROWSER UI

Project: MonkDB Workbench **Module:** Full-Text Search (FTS) **Report Date:** 2026-03-02 **Prepared by:** Engineering Team **Status:** ✅ All 68 tests passing

Executive Summary

This report documents the end-to-end test suite written for the Full-Text Search module of the MonkDB Workbench. The suite comprises **68 automated tests** split across two projects:

Suite	Focus	Tests	Result
`sql-compat`	MonkDB SQL API compatibility (no browser)	37	✅ 37 passed
`ui-fts`	Browser UI end-to-end (Chromium)	31	✅ 31 passed
Total		68	✅ **68 passed, 0 failed**

Tests run in parallel workers. Combined wall-clock time: **~1 minute 10 seconds**.

In addition to confirming correct behaviour, testing against a live MonkDB 6.0 cluster revealed **3 important engine limitations** that are now documented and reflected in the application UI (see Key Findings).

Test Environment

Component	Version
MonkDB	6.0.0-SNAPSHOT (Lucene 10.1.0)
Node.js	25.2.1
Playwright	1.58.2
Browser	Chromium (headless + headed)
Next.js dev server	`http://localhost:3000`
MonkDB HTTP API	`http://localhost:4200/_sql`

How to Run

```
# Install dependencies (first time only)
npx playwright install chromium

# Run the full suite (both projects, parallel workers)
npx playwright test

# Run SQL-only tests (no browser required)
npx playwright test --project=sql-compat

# Run browser UI tests (headless)
npx playwright test --project=ui-fts

# Run browser UI tests in headed mode (watch the browser)
npx playwright test --project=ui-fts --headed

# Open the interactive HTML report after any run
npx playwright show-report
```

Prerequisites: MonkDB must be running on `localhost:4200` and the Next.js dev server on `localhost:3000` (auto-started by Playwright's `webServer` config if not already running).

Suite 1 — SQL Compatibility (`sql-compat`)

Tests every SQL query used by the FTS module by posting directly to the MonkDB `/_sql` HTTP endpoint. No browser is involved. This suite proves that each query the application code depends on is supported by MonkDB 6.0 with the correct syntax.

File: `e2e/fts/sql-compat.spec.ts`

Test Table Setup

Each run creates a dedicated test table `doc.pw_fts_test` with a named `FULLTEXT` index, inserts 5 rows, and calls `REFRESH TABLE`. The table is dropped in `afterAll`.

```
CREATE TABLE "doc"."pw_fts_test" (
  id     INTEGER PRIMARY KEY,
  title  TEXT,
  body   TEXT,
  author TEXT,
  INDEX "idx_pw_fts_test" USING FULLTEXT (title, body) WITH (analyzer = 'english')
);
```

Test Cases

GROUP 1 — TABLE DISCOVERY

ID	Test Name	What it verifies
01	`information_schema.tables` returns BASE TABLE rows	The query used by `useFTSIndexes` to list user tables returns rows with correct columns; our test table appears
01b	`sys` / `information_schema` / `pg_catalog` are excluded	The `NOT IN (...)` filter correctly removes system schemas from the table list

GROUP 2 — DDL INSPECTION

ID	Test Name	What it verifies
02	`SHOW CREATE TABLE` returns DDL string	Returns exactly 1 row; content is a non-empty string
02b	DDL contains named FULLTEXT index	The regex `INDEX\s+"([^"]+)"\s+USING\s+FULLTEXT\s+\(([^)]+)\)` (identical to production code) finds the index, columns, and analyzer

ID	Test Name	What it verifies
02c	`SHOW CREATE TABLE` on non-existent table returns error	Confirms error-handling path; MonkDB returns a structured error response

GROUP 3 — ROW COUNT

ID	Test Name	What it verifies
03	`SELECT COUNT(*)` returns numeric row count	Returns exactly 5 (matches inserted rows); `parseInt(..., 10)` parse works correctly

GROUP 4 — COLUMN METADATA

ID	Test Name	What it verifies
04	`information_schema.columns` works with `?` parameters	`WHERE table_schema = ? AND table_name = ?` returns the correct columns
04b	`data_type` for TEXT column is `"text"`	MonkDB uses lowercase `text` (not `varchar`, `character varying`, etc.)
04c	`data_type` for INTEGER column is `"integer"`	MonkDB uses lowercase `integer`

GROUP 5 — PRIMARY KEY DETECTION

ID	Test Name	What it verifies
05	PK JOIN query returns primary key columns	The `key_column_usage JOIN table_constraints` query correctly identifies `id` as the PK
05b	`constraint_type = 'PRIMARY KEY'` works	MonkDB exposes `PRIMARY KEY` as `constraint_type` (not the PostgreSQL `_pkey` suffix pattern)

GROUP 6 — PARAMETER SYNTAX

ID	Test Name	What it verifies
06	MonkDB uses `?` placeholders	`WHERE table_name = ?` with `args: [value]` works correctly
06b	PostgreSQL-style `$1` placeholder fails	`$1` is treated as a literal string — returns 0 rows or an error; confirms `?` is the required syntax

GROUP 7 — REFRESH TABLE

ID	Test Name	What it verifies
07	`REFRESH TABLE` executes without error	Returns `rowcount ≥ 0` (number of shards refreshed)

GROUP 8 — MATCH FULL-TEXT SEARCH

ID	Test Name	What it verifies
08	`MATCH("index_name", ?)` returns results with `_score`	The correct MonkDB 6+ index-name syntax works; `_score` column is present and numeric
08b	Results ranked by `_score` descending	Scores are non-increasing when `ORDER BY _score DESC` is used
08c	`"error"` query matches rows 1 and 4	Both rows containing the word "error" are returned
08d	Phrase query `"connection timeout"` matches row 1	Exact phrase search works as expected
08e	Minus-exclude is NOT enforced ⚠️	`"error -distributed"` still returns row 4 (see Key Findings)
08f	Wildcard `connect*` matches row 1	Prefix wildcard works when using the stemmed root form
08g	`LIMIT` clause is respected	Result count does not exceed the specified limit

GROUP 9 — LEGACY COLUMN SYNTAX (NEGATIVE TEST)

ID	Test Name	What it verifies
09	`MATCH(column_name, ?)` returns 0 results	The old column-based MATCH syntax is broken in MonkDB 6+; confirms our migration to index-name syntax is necessary

GROUP 10 — CREATE / INSERT / REFRESH CYCLE

ID	Test Name	What it verifies
10	`CREATE TABLE` with FULLTEXT index succeeds	Full DDL round-trip works; table appears in `information_schema`
10b	`INSERT INTO … SELECT * FROM` copies all 5 rows	Bulk data copy used during index re-creation works
10c	MATCH on copy table works after REFRESH	End-to-end re-index flow is functional

ID	Test Name	What it verifies
10d	MATCH returns 0 rows before REFRESH	**Confirms MonkDB's eventual-consistency model; REFRESH is mandatory for MATCH visibility**

GROUP 11 — BATCH SHOW CREATE TABLE

ID	Test Name	What it verifies
11	10 concurrent `SHOW CREATE TABLE` calls succeed	**Simulates `batchedMap(tables, 10, ...)` in `useFTSIndexes.ts`; all non-BLOB tables succeed**

> **Note:** BLOB tables (schema `blob`) throw `OperationOnInaccessibleRelationException` for `SHOW CREATE TABLE`. The `blob` schema is now excluded from the table discovery query.

GROUP 12 — SYSTEM SCHEMA ACCESSIBILITY

ID	Test Name	What it verifies
12	`sys` schema is queryable	**`SELECT name FROM sys.cluster` works; confirms sys is correctly excluded from FTS list but is accessible**
12b	`information_schema` is queryable	**Contains `doc`, `sys`, `information_schema` in `schemata`**
12c	`pg_catalog` exists in MonkDB	**Confirmed present; exclusion in `useFTSIndexes` is correct and necessary**

GROUP 13 — `_SCORE` COLUMN BEHAVIOUR

ID	Test Name	What it verifies
13	`SELECT *` does NOT include `_score`	**`_score` is a virtual column; must be requested explicitly**
13b	`SELECT *, _score` includes it	**Explicit selection works; value is a positive float**

GROUP 14 — QUOTING & ESCAPING

ID	Test Name	What it verifies
14	`REFRESH TABLE` with quoted `"schema"."table"` succeeds	**Double-quote identifier quoting is correct**
14b	`REFRESH TABLE` on non-existent table returns error	**Error path is a structured MonkDB error**

ID	Test Name	What it verifies
15	`buildMatchQuery` double-quote escaping	Index name with quotes is escaped via `.replace(/"/g, '""')` before injection into `MATCH("...", ?)`

GROUP 16 — EDGE CASES

ID	Test Name	What it verifies
16	MATCH with non-existent term returns 0 rows	`xyznonexistentterm12345` produces no results
16b	Multi-term query uses OR semantics	`"error BM25"` returns rows 1, 3, and 4 (union of individual matches)

Suite 2 — Browser UI (`ui-fts`)

Tests the FTS page (`/fts`) running in a live Chromium browser against the real Next.js dev server and real MonkDB. Each test navigates to the page with a pre-injected localStorage connection so the app connects to the local MonkDB instance automatically.

File: `e2e/fts/ui-fts.spec.ts`

Connection Injection

Because the app reads connection credentials from `localStorage`, every test injects them before the page loads:

```
await page.addInitScript(([payload, id]) => {
  localStorage.setItem('monkdb_connections', payload);
  localStorage.setItem('monkdb_active_connection', id);
}, [CONN_PAYLOAD, CONN_ID]);
```

This simulates a logged-in user without going through the connection dialog.

Test Table

The UI tests create their own dedicated table `doc.pw_fts_ui` (separate from the sql-compat table) so the two suites can run in parallel without conflict. The table is always dropped and recreated in `beforeAll` and cleaned up in `afterAll`.

Test Cases

GROUP 1 — PAGE LOAD

ID	Test Name	What it verifies
01	FTS page loads and shows "Full-Text Search" title	**Page renders correctly; main heading is visible**
01b	Left panel search box is visible	**`input[placeholder="Filter indexes…"]` is rendered**
01c	Search textarea not visible before table selected	**The search area only appears after selecting an index from the left panel**

GROUP 2 — INDEX BROWSER (LEFT PANEL)

ID	Test Name	What it verifies
02	Left panel shows test FTS table after indexes load	**`pw_fts_ui` appears as a clickable button in the left panel within 20 seconds**
02b	"FTS Indexes" label is visible	**Section heading renders in the left panel**

GROUP 3 — INDEX SELECTION

ID	Test Name	What it verifies
03	Clicking test table shows indexed columns	**After selection, `title` and/or `body` column names appear in the right panel**
03b	Schema name appears in selected index header	**`doc` schema label is visible after selection**

GROUP 4 — SEARCH EXECUTION

ID	Test Name	What it verifies
04	Search textarea has correct placeholder	**Placeholder contains "search query"**
04b	Searching "error" returns result rows	**Row titles containing "error" are displayed in results**
04c	Results counter shows N results	**Toolbar displays a count matching pattern `\d+ result(s)`**

GROUP 5 — BM25 SCORING

ID	Test Name	What it verifies
05	BM25 score displayed (4 decimal places)	Result rows show a score formatted as `X.XXXX`

GROUP 6 — TERM HIGHLIGHTING

ID	Test Name	What it verifies
06	Matched terms wrapped in `<mark>` elements	`dangerouslySetInnerHTML` output contains `<mark>` tags; text inside contains "error"

GROUP 7 — SQL PREVIEW

ID	Test Name	What it verifies
07	SQL preview toggle button is visible	`button[title="Toggle SQL preview"]` is rendered after index selection
07b	SQL preview shows correct index-name MATCH syntax	Preview contains `MATCH` + the index name; does NOT use old column-name syntax `MATCH(title, ?)`

GROUP 8 — EXPORT BUTTONS

ID	Test Name	What it verifies
08	CSV and JSON export buttons appear after search	Both buttons visible in results toolbar
08b	Copy button appears in results toolbar	Clipboard copy action is available

GROUP 9 — CREATE FTS INDEX WIZARD

ID	Test Name	What it verifies
09	"New FTS Index" button is in the header	Button is visible on initial page load
09b	Clicking it opens the wizard	Step 1 / "Create Full-Text Search Table" heading appears
09c	Wizard shows a schema select dropdown	`<select>` element is visible in step 1
09d	Cancel closes the wizard	Wizard disappears; page title is still visible

GROUP 10 — QUERY SYNTAX TIPS PANEL

ID	Test Name	What it verifies
10	"Query Syntax" section is visible	Right panel shows the syntax tips heading

ID	Test Name	What it verifies
10b	Single Term, Phrase, Boolean OR tips shown	All three built-in tip labels are present
10c	"Must Exclude" tip is NOT present	Removed because MonkDB 6.0 does not enforce `-term` exclusion
10d	Prefix Match example shows `connect*`	Corrected from `conn*` which fails with the English stemming analyzer

GROUP 11 — USER QUERY SNIPPETS

ID	Test Name	What it verifies
11	"Add Snippet" button is visible	Right panel shows the button
11b	Clicking it shows label and query inputs	Form renders with `input[placeholder*="Label"]` and `input[placeholder*="Query"]`
11c	Can save a snippet and see it in the list	After filling and clicking Save, the label appears in the snippet list
11d	Snippet persists after page reload	Snippet saved to localStorage survives a `page.reload()`

GROUP 12 — NETWORK / SQL CORRECTNESS

ID	Test Name	What it verifies
12	No `$1` / `$2` PostgreSQL placeholders in any SQL	Intercepts all `/api/monkdb/query` requests; confirms `?` syntax used throughout
12b	No unhandled JS errors on the FTS page	Page error listener confirms zero uncaught exceptions during load and interaction

GROUP 13 — REFRESH

ID	Test Name	What it verifies
13	Header "Refresh" button reloads index list	Click does not crash; page remains functional after refresh

Key Findings

Testing against a live MonkDB 6.0.0-SNAPSHOT cluster revealed the following behaviours. These findings have been incorporated into the application UI and documentation.

Finding 1 — Minus Exclusion Not Enforced

Query: `MATCH("idx", ?) → 'error -distributed'` **Expected:** Only rows containing "error" but not "distributed" **Actual:** All rows containing "error" are returned; `-distributed` is silently ignored

Impact: The "Must Exclude" syntax tip has been **removed** from the Query Syntax panel in the UI. Users should not rely on minus-prefix exclusion.

Test: `sql-compat › 08e`

Finding 2 — Wildcard Must Use Stemmed Form

Query (broken): `MATCH("idx", ?) → 'conn*'` **Query (correct):** `MATCH("idx", ?) → 'connect*'`

With the `english` analyzer, the word "connection" is stemmed to "connect" at index time. A wildcard prefix must match the stemmed root, not the original word prefix.

Impact: The Prefix Match example in the Query Syntax panel was corrected from `conn*` to `connect*`.

Test: `sql-compat › 08f`

Finding 3 — BLOB Schema Tables Inaccessible

Query: `SHOW CREATE TABLE "blob"."<table>"` **Error:** `OperationOnInaccessibleRelationException`

BLOB tables appear as `BASE TABLE` in `information_schema.tables` but `SHOW CREATE TABLE` throws an exception for them. They cannot be distinguished from regular tables by type alone.

Impact: The `blob` schema was added to the exclusion list in `useFTSIndexes.ts`:

```
WHERE table_type = 'BASE TABLE'
  AND table_schema NOT IN ('sys', 'information_schema', 'pg_catalog', 'blob')
```

Test: `sql-compat › 11`

Coverage Map

Application Layer	Coverage
`useFTSIndexes.ts` — table discovery query	✅ Tests 01, 01b
`useFTSIndexes.ts` — SHOW CREATE TABLE + regex parsing	✅ Tests 02, 02b, 11
`useFTSIndexes.ts` — SELECT COUNT(*)	✅ Test 03
`useFTSIndexes.ts` — blob schema exclusion	✅ Test 11
`useFTSIndexes.ts` — batchedMap concurrency	✅ Test 11
`fts-utils.ts` — buildMatchQuery (index-name syntax)	✅ Tests 08, 08b–08g, UI 07b
`fts-utils.ts` — double-quote escaping in index names	✅ Test 15
`fts-utils.ts` — REFRESH TABLE	✅ Tests 07, 14, 10d
`CreateFTSIndexDialog.tsx` — PK detection query	✅ Tests 05, 05b
`CreateFTSIndexDialog.tsx` — column metadata query	✅ Tests 04, 04b, 04c
`CreateFTSIndexDialog.tsx` — CREATE TABLE + INSERT + REFRESH	✅ Tests 10, 10b, 10c
Parameter syntax (`?` not `$1`)	✅ Tests 06, 06b, UI 12
`_score` virtual column behaviour	✅ Tests 13, 13b, UI 05
Query Syntax tips accuracy vs. MonkDB 6.0	✅ UI Tests 10b–10d
User query snippets (localStorage)	✅ UI Tests 11–11d
Term highlighting (`<mark>`)	✅ UI Test 06
SQL preview (correct MATCH syntax)	✅ UI Test 07b
Export buttons (CSV, JSON, Copy)	✅ UI Tests 08, 08b
Create FTS Index wizard flow	✅ UI Tests 09–09d
No JS errors on page load	✅ UI Test 12b

File Reference

File	Purpose
`e2e/fts/sql-compat.spec.ts`	SQL compatibility tests (37 tests)
`e2e/fts/ui-fts.spec.ts`	Browser UI tests (31 tests)
`playwright.config.ts`	Playwright configuration (projects, webServer, timeouts)
`docs/full-text-search-guide.md`	Developer guide for the FTS feature
`app/hooks/useFTSIndexes.ts`	Hook: discovers FTS tables via SHOW CREATE TABLE
`app/lib/fts-utils.ts`	Core: buildMatchQuery, REFRESH, export helpers
`app/fts/page.tsx`	FTS page: search UI, syntax tips, SQL snippets